1

Introduction

Over the past two decades Machine Learning has become one of the mainstays of information technology and with that, a rather central, albeit usually hidden, part of our life. With the ever increasing amounts of data becoming available there is good reason to believe that smart data analysis will become even more pervasive as a necessary ingredient for technological progress.

The purpose of this chapter is to provide the reader with an overview over the vast range of applications which have at their heart a machine learning problem and to bring some degree of order to the zoo of problems. After that, we will discuss some basic tools from statistics and probability theory, since they form the language in which many machine learning problems must be phrased to become amenable to solving. Finally, we will outline a set of fairly basic yet effective algorithms to solve an important problem, namely that of classification. More sophisticated tools, a discussion of more general problems and a detailed analysis will follow in later parts of the book.

1.1 A Taste of Machine Learning

Machine learning can appear in many guises. We now discuss a number of applications, the types of data they deal with, and finally, we formalize the problems in a somewhat more stylized fashion. The latter is key if we want to avoid reinventing the wheel for every new application. Instead, much of the *art* of machine learning is to reduce a range of fairly disparate problems to a set of fairly narrow prototypes. Much of the *science* of machine learning is then to solve those problems and provide good guarantees for the solutions.

1.1.1 Applications

Most readers will be familiar with the concept of web page **ranking**. That is, the process of submitting a query to a search engine, which then finds webpages relevant to the query and which returns them in their order of relevance. See e.g. Figure 1.1 for an example of the query results for "machine learning". That is, the search engine returns a sorted list of webpages given a query. To achieve this goal, a search engine needs to 'know' which

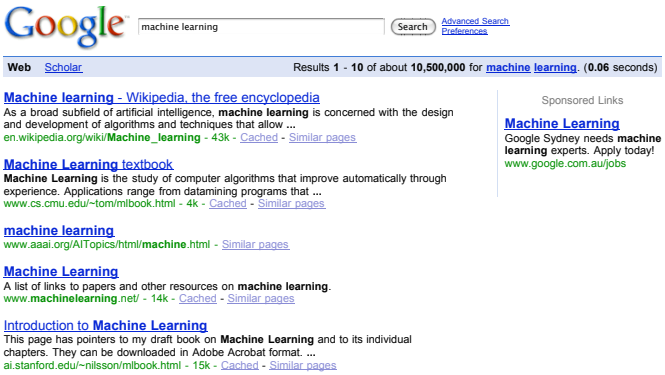

Fig. 1.1. The 5 top scoring webpages for the query "machine learning"

pages are relevant and which pages match the query. Such knowledge can be gained from several sources: the link structure of webpages, their content, the frequency with which users will follow the suggested links in a query, or from examples of queries in combination with manually ranked webpages. Increasingly machine learning rather than guesswork and clever engineering is used to *automate* the process of designing a good search engine [RPB06].

A rather related application is **collaborative filtering**. Internet bookstores such as Amazon, or video rental sites such as Netflix use this information extensively to entice users to purchase additional goods (or rent more movies). The problem is quite similar to the one of web page ranking. As before, we want to obtain a sorted list (in this case of articles). The key difference is that an explicit query is missing and instead we can only use past purchase and viewing decisions of the user to predict future viewing and purchase habits. The key side information here are the decisions made by *similar* users, hence the collaborative nature of the process. See Figure 1.2 for an example. It is clearly desirable to have an automatic system to solve this problem, thereby avoiding guesswork and time [BK07].

An equally ill-defined problem is that of **automatic translation** of documents. At one extreme, we could aim at fully *understanding* a text before translating it using a curated set of rules crafted by a computational linguist well versed in the two languages we would like to translate. This is a rather arduous task, in particular given that text is not always grammatically correct, nor is the document understanding part itself a trivial one. Instead, we could simply use *examples* of translated documents, such as the proceedings of the Canadian parliament or other multilingual entities (United Nations, European Union, Switzerland) to *learn* how to translate between the two

languages. In other words, we could use examples of translations to learn how to translate. This machine learning approach proved quite successful [**?**].

Many security applications, e.g. for access control, use face recognition as one of its components. That is, given the photo (or video recording) of a person, recognize who this person is. In other words, the system needs to **classify** the faces into one of many categories (Alice, Bob, Charlie, ...) or decide that it is an unknown face. A similar, yet conceptually quite different problem is that of verification. Here the goal is to verify whether the person in question is who he claims to be. Note that differently to before, this is now a yes/no question. To deal with different lighting conditions, facial expressions, whether a person is wearing glasses, hairstyle, etc., it is desirable to have a system which *learns* which features are relevant for identifying a person.

Another application where learning helps is the problem of **named entity recognition** (see Figure 1.4). That is, the problem of identifying entities, such as places, titles, names, actions, etc. from documents. Such steps are crucial in the automatic digestion and understanding of documents. Some modern e-mail clients, such as Apple's Mail.app nowadays ship with the ability to identify addresses in mails and filing them automatically in an address book. While systems using hand-crafted rules can lead to satisfactory results, it is far more efficient to use examples of marked-up documents to learn such dependencies automatically, in particular if we want to deploy our system in many languages. For instance, while 'bush' and 'rice'

Fig. 1.2. Books recommended by Amazon.com when viewing Tom Mitchell's Machine Learning Book [Mit97]. It is desirable for the vendor to recommend relevant books which a user might purchase.

Fig. 1.3. 11 Pictures of the same person taken from the Yale face recognition database. The challenge is to recognize that we are dealing with the same person in all 11 cases.

```
HAVANA (Reuters) - The European Union's top development aid official
left Cuba on Sunday convinced that EU diplomatic sanctions against
the communist island should be dropped after Fidel Castro's
retirement, his main aide said.
```

```
<TYPE="ORGANIZATION">HAVANA</> (<TYPE="ORGANIZATION">Reuters</>) - The
<TYPE="ORGANIZATION">European Union</>'s top development aid official left
<TYPE="ORGANIZATION">Cuba</> on Sunday convinced that EU diplomatic sanctions
against the communist <TYPE="LOCATION">island</> should be dropped after
<TYPE="PERSON">Fidel Castro</>'s retirement, his main aide said.
```

Fig. 1.4. Named entity tagging of a news article (using LingPipe). The relevant locations, organizations and persons are tagged for further information extraction.

are clearly terms from agriculture, it is equally clear that in the context of contemporary politics they refer to members of the Republican Party.

Other applications which take advantage of learning are **speech recognition** (annotate an audio sequence with text, such as the system shipping with Microsoft Vista), the recognition of handwriting (annotate a sequence of strokes with text, a feature common to many PDAs), trackpads of computers (e.g. Synaptics, a major manufacturer of such pads derives its name from the synapses of a neural network), the detection of failure in jet engines, avatar behavior in computer games (e.g. Black and White), direct marketing (companies use past purchase behavior to guesstimate whether you might be willing to purchase even more) and floor cleaning robots (such as iRobot's Roomba). The overarching theme of learning problems is that there exists a nontrivial dependence between some observations, which we will commonly refer to as x and a desired response, which we refer to as y, for which a simple set of deterministic rules is not known. By using learning we can infer such a dependency between x and y in a systematic fashion.

We conclude this section by discussing the problem of **classification**, since it will serve as a prototypical problem for a significant part of this book. It occurs frequently in practice: for instance, when performing spam filtering, we are interested in a yes/no answer as to whether an e-mail contains relevant information or not. Note that this issue is quite user dependent: for a frequent traveller e-mails from an airline informing him about recent discounts might prove valuable information, whereas for many other recipients this might prove more of an nuisance (e.g. when the e-mail relates to products available only overseas). Moreover, the nature of annoying e-mails might change over time, e.g. through the availability of new products (Viagra, Cialis, Levitra, ...), different opportunities for fraud (the Nigerian 419 scam which took a new twist after the Iraq war), or different data types (e.g. spam which consists mainly of images). To combat these problems we

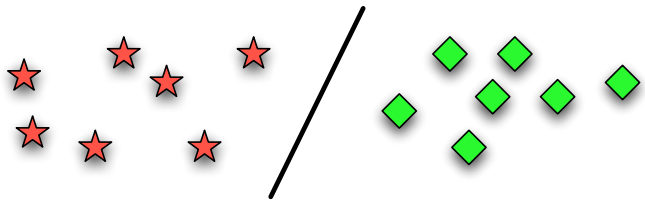

Fig. 1.5. Binary classification; separate stars from diamonds. In this example we are able to do so by drawing a straight line which separates both sets. We will see later that this is an important example of what is called a *linear classifier*.

want to build a system which is able to *learn* how to classify new e-mails. A seemingly unrelated problem, that of cancer diagnosis shares a common structure: given histological data (e.g. from a microarray analysis of a patient's tissue) infer whether a patient is healthy or not. Again, we are asked to generate a yes/no answer given a set of observations. See Figure 1.5 for an example.

1.1.2 Data

It is useful to characterize learning problems according to the type of data they use. This is a great help when encountering new challenges, since quite often problems on similar data types can be solved with very similar techniques. For instance natural language processing and bioinformatics use very similar tools for strings of natural language text and for DNA sequences. **Vectors** constitute the most basic entity we might encounter in our work. For instance, a life insurance company might be interesting in obtaining the vector of variables (blood pressure, heart rate, height, weight, cholesterol level, smoker, gender) to infer the life expectancy of a potential customer. A farmer might be interested in determining the ripeness of fruit based on (size, weight, spectral data). An engineer might want to find dependencies in (voltage, current) pairs. Likewise one might want to represent documents by a vector of counts which describe the occurrence of words. The latter is commonly referred to as bag of words features.

One of the challenges in dealing with vectors is that the *scales* and units of different coordinates may vary widely. For instance, we could measure the height in kilograms, pounds, grams, tons, stones, all of which would amount to multiplicative changes. Likewise, when representing temperatures, we have a full class of affine transformations, depending on whether we represent them in terms of Celsius, Kelvin or Farenheit. One way of dealing

with those issues in an automatic fashion is to normalize the data. We will discuss means of doing so in an automatic fashion.

Lists: In some cases the vectors we obtain may contain a variable number of features. For instance, a physician might not necessarily decide to perform a full battery of diagnostic tests if the patient appears to be healthy.

Sets may appear in learning problems whenever there is a large number of potential causes of an effect, which are not well determined. For instance, it is relatively easy to obtain data concerning the toxicity of mushrooms. It would be desirable to use such data to infer the toxicity of a new mushroom given information about its chemical compounds. However, mushrooms contain a cocktail of compounds out of which one or more may be toxic. Consequently we need to infer the properties of an object given a *set* of features, whose composition and number may vary considerably.

Matrices are a convenient means of representing pairwise relationships. For instance, in collaborative filtering applications the rows of the matrix may represent users whereas the columns correspond to products. Only in some cases we will have knowledge about a given (user, product) combination, such as the rating of the product by a user.

A related situation occurs whenever we only have similarity information between observations, as implemented by a semi-empirical distance measure. Some homology searches in bioinformatics, e.g. variants of BLAST [AGML90], only return a similarity score which does not necessarily satisfy the requirements of a metric.

Images could be thought of as two dimensional arrays of numbers, that is, matrices. This representation is very crude, though, since they exhibit spatial coherence (lines, shapes) and (natural images exhibit) a multiresolution structure. That is, downsampling an image leads to an object which has very similar statistics to the original image. Computer vision and psychooptics have created a raft of tools for describing these phenomena.

Video adds a temporal dimension to images. Again, we could represent them as a three dimensional array. Good algorithms, however, take the temporal coherence of the image sequence into account.

Trees and Graphs are often used to describe relations between collections of objects. For instance the ontology of webpages of the DMOZ project (`www.dmoz.org`) has the form of a tree with topics becoming increasingly refined as we traverse from the root to one of the leaves (Arts → Animation → Anime → General Fan Pages → Official Sites). In the case of gene ontology the relationships form a directed acyclic graph, also referred to as the GO-DAG [ABB+00].

Both examples above describe estimation problems where our observations

are vertices of a tree or graph. However, graphs themselves may be the observations. For instance, the DOM-tree of a webpage, the call-graph of a computer program, or the protein-protein interaction networks may form the basis upon which we may want to perform inference.

Strings occur frequently, mainly in the area of bioinformatics and natural language processing. They may be the input to our estimation problems, e.g. when classifying an e-mail as spam, when attempting to locate all names of persons and organizations in a text, or when modeling the topic structure of a document. Equally well they may constitute the output of a system. For instance, we may want to perform document summarization, automatic translation, or attempt to answer natural language queries.

Compound structures are the most commonly occurring object. That is, in most situations we will have a structured mix of different data types. For instance, a webpage might contain images, text, tables, which in turn contain numbers, and lists, all of which might constitute nodes on a graph of webpages linked among each other. Good statistical modelling takes such dependencies and structures into account in order to tailor sufficiently flexible models.

1.1.3 Problems

The range of learning problems is clearly large, as we saw when discussing applications. That said, researchers have identified an ever growing number of templates which can be used to address a large set of situations. It is those templates which make deployment of machine learning in practice easy and our discussion will largely focus on a choice set of such problems. We now give a by no means complete list of templates.

Binary Classification is probably the most frequently studied problem in machine learning and it has led to a large number of important algorithmic and theoretic developments over the past century. In its simplest form it reduces to the question: given a pattern x drawn from a domain $\mathcal{X}$, estimate which value an associated binary random variable $y \in \{\pm 1\}$ will assume. For instance, given pictures of apples and oranges, we might want to state whether the object in question is an apple or an orange. Equally well, we might want to predict whether a home owner might default on his loan, given income data, his credit history, or whether a given e-mail is spam or ham. The ability to solve this basic problem already allows us to address a large variety of practical settings.

There are many variants exist with regard to the protocol in which we are required to make our estimation:

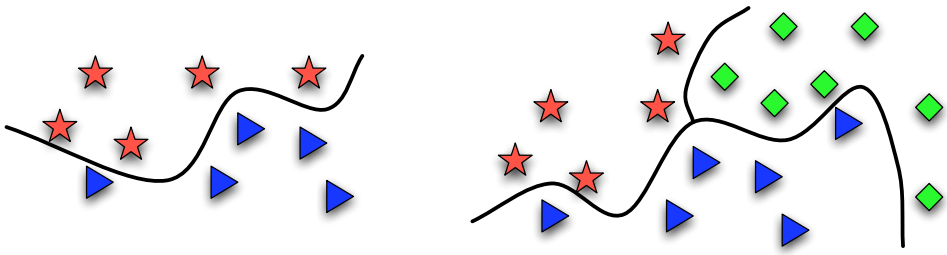

Fig. 1.6. Left: binary classification. Right: 3-class classification. Note that in the latter case we have much more degree for ambiguity. For instance, being able to distinguish stars from diamonds may not suffice to identify either of them correctly, since we also need to distinguish both of them from triangles.

- We might see a sequence of (x_i, y_i) pairs for which y_i needs to be estimated in an instantaneous online fashion. This is commonly referred to as online learning.
- We might observe a collection $\mathbf{X} := \{x_1, \dots x_m\}$ and $\mathbf{Y} := \{y_1, \dots y_m\}$ of pairs (x_i, y_i) which are then used to estimate y for a (set of) so-far unseen $\mathbf{X}' = \{x'_1, \dots, x'_{m'}\}$. This is commonly referred to as batch learning.
- We might be allowed to know $\mathbf{X}'$ already at the time of constructing the model. This is commonly referred to as transduction.
- We might be allowed to choose $\mathbf{X}$ for the purpose of model building. This is known as active learning.
- We might not have full information about $\mathbf{X}$, e.g. some of the coordinates of the x_i might be missing, leading to the problem of estimation with missing variables.
- The sets $\mathbf{X}$ and $\mathbf{X}'$ might come from different data sources, leading to the problem of covariate shift correction.
- We might be given observations stemming from two problems at the same time with the side information that both problems are somehow related. This is known as co-training.
- Mistakes of estimation might be penalized differently depending on the type of error, e.g. when trying to distinguish diamonds from rocks a very asymmetric loss applies.

Multiclass Classification is the logical extension of binary classification. The main difference is that now $y \in \{1, \dots, n\}$ may assume a range of different values. For instance, we might want to classify a document according to the language it was written in (English, French, German, Spanish, Hindi, Japanese, Chinese, ...). See Figure 1.6 for an example. The main difference to before is that the cost of error may heavily depend on the type of

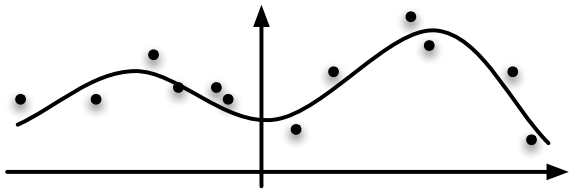

Fig. 1.7. Regression estimation. We are given a number of instances (indicated by black dots) and would like to find some function f mapping the observations $\mathcal{X}$ to $\mathbb{R}$ such that $f(x)$ is close to the observed values.

error we make. For instance, in the problem of assessing the risk of cancer, it makes a significant difference whether we mis-classify an early stage of cancer as healthy (in which case the patient is likely to die) or as an advanced stage of cancer (in which case the patient is likely to be inconvenienced from overly aggressive treatment).

Structured Estimation goes beyond simple multiclass estimation by assuming that the labels y have some additional structure which can be used in the estimation process. For instance, y might be a path in an ontology, when attempting to classify webpages, y might be a permutation, when attempting to match objects, to perform collaborative filtering, or to rank documents in a retrieval setting. Equally well, y might be an annotation of a text, when performing named entity recognition. Each of those problems has its own properties in terms of the set of y which we might consider admissible, or how to search this space. We will discuss a number of those problems in Chapter **??**.

Regression is another prototypical application. Here the goal is to estimate a real-valued variable $y \in \mathbb{R}$ given a pattern x (see e.g. Figure 1.7). For instance, we might want to estimate the value of a stock the next day, the yield of a semiconductor fab given the current process, the iron content of ore given mass spectroscopy measurements, or the heart rate of an athlete, given accelerometer data. One of the key issues in which regression problems differ from each other is the choice of a loss. For instance, when estimating stock values our loss for a put option will be decidedly one-sided. On the other hand, a hobby athlete might only care that our estimate of the heart rate matches the actual on average.

Novelty Detection is a rather ill-defined problem. It describes the issue of determining "unusual" observations given a set of past measurements. Clearly, the choice of what is to be considered unusual is very subjective. A commonly accepted notion is that unusual events occur rarely. Hence a possible goal is to design a system which assigns to each observation a rating

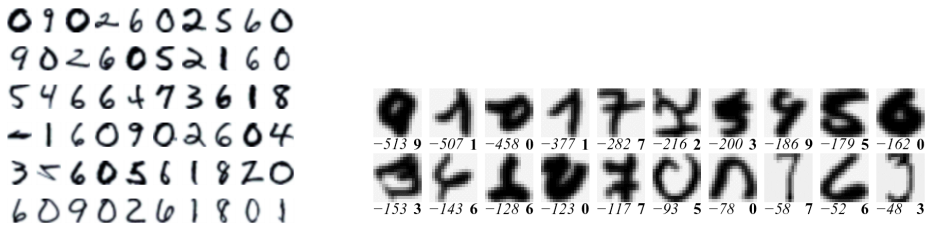

Fig. 1.8. Left: typical digits contained in the database of the US Postal Service. Right: unusual digits found by a novelty detection algorithm [SPST+01] (for a description of the algorithm see Section 7.4). The score below the digits indicates the degree of novelty. The numbers on the lower right indicate the class associated with the digit.

as to how novel it is. Readers familiar with density estimation might contend that the latter would be a reasonable solution. However, we neither need a score which sums up to 1 on the entire domain, nor do we care particularly much about novelty scores for *typical* observations. We will later see how this somewhat easier goal can be achieved directly. Figure 1.8 has an example of novelty detection when applied to an optical character recognition database.

1.2 Probability Theory

In order to deal with the instances of where machine learning can be used, we need to develop an adequate language which is able to describe the problems concisely. Below we begin with a fairly informal overview over probability theory. For more details and a very gentle and detailed discussion see the excellent book of [BT03].

1.2.1 Random Variables

Assume that we cast a dice and we would like to know our chances whether we would see 1 rather than another digit. If the dice is fair all six outcomes $\mathcal{X} = \{1, \ldots, 6\}$ are equally likely to occur, hence we would see a 1 in roughly 1 out of 6 cases. Probability theory allows us to model uncertainty in the outcome of such experiments. Formally we state that 1 occurs with probability $\frac{1}{6}$.

In many experiments, such as the roll of a dice, the outcomes are of a numerical nature and we can handle them easily. In other cases, the outcomes may not be numerical, *e.g.,* if we toss a coin and observe heads or tails. In these cases, it is useful to associate numerical values to the outcomes. This is done via a random variable. For instance, we can let a random variable

X take on a value $+1$ whenever the coin lands heads and a value of -1 otherwise. Our notational convention will be to use uppercase letters, *e.g.,* X, Y etc to denote random variables and lower case letters, *e.g.,* x, y etc to denote the values they take.

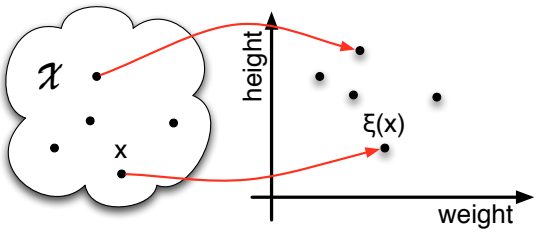

Fig. 1.9. The random variable ξ maps from the set of outcomes of an experiment (denoted here by $\mathcal{X}$) to real numbers. As an illustration here $\mathcal{X}$ consists of the patients a physician might encounter, and they are mapped via ξ to their weight and height.

1.2.2 Distributions

Perhaps the most important way to characterize a random variable is to associate probabilities with the values it can take. If the random variable is discrete, *i.e.,* it takes on a finite number of values, then this assignment of probabilities is called a *probability mass function* or PMF for short. A PMF must be, by definition, non-negative and must sum to one. For instance, if the coin is fair, *i.e.,* heads and tails are equally likely, then the random variable X described above takes on values of $+1$ and -1 with probability 0.5. This can be written as

$$Pr(X = +1) = 0.5 \text{ and } Pr(X = -1) = 0.5. \tag{1.1}$$

When there is no danger of confusion we will use the slightly informal notation $p(x) := Pr(X = x)$.

In case of a continuous random variable the assignment of probabilities results in a *probability density function* or PDF for short. With some abuse of terminology, but keeping in line with convention, we will often use density or distribution instead of probability density function. As in the case of the PMF, a PDF must also be non-negative and integrate to one. Figure 1.10 shows two distributions: the uniform distribution

$$p(x) = \begin{cases} \frac{1}{b-a} & \text{if } x \in [a, b] \\ 0 & \text{otherwise,} \end{cases} \tag{1.2}$$

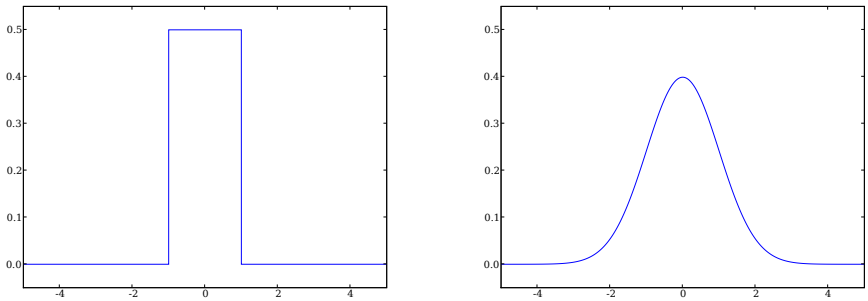

Fig. 1.10. Two common densities. Left: uniform distribution over the interval $[-1, 1]$. Right: Normal distribution with zero mean and unit variance.

and the Gaussian distribution (also called normal distribution)

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right). \tag{1.3}$$

Closely associated with a PDF is the indefinite integral over p. It is commonly referred to as the cumulative distribution function (CDF).

Definition 1.1 (Cumulative Distribution Function) *For a real valued random variable X with PDF p the associated Cumulative Distribution Function F is given by*

$$F(x') := \Pr\left\{X \leq x'\right\} = \int_{-\infty}^{x'} dp(x). \tag{1.4}$$

The CDF $F(x')$ allows us to perform range queries on p efficiently. For instance, by integral calculus we obtain

$$\Pr(a \leq X \leq b) = \int_a^b dp(x) = F(b) - F(a). \tag{1.5}$$

The values of x' for which $F(x')$ assumes a specific value, such as 0.1 or 0.5 have a special name. They are called the *quantiles* of the distribution p.

Definition 1.2 (Quantiles) *Let $q \in (0, 1)$. Then the value of x' for which $\Pr(X < x') \leq q$ and $\Pr(X > x') \leq 1 - q$ is the q-quantile of the distribution p. Moreover, the value x' associated with $q = 0.5$ is called the median.*

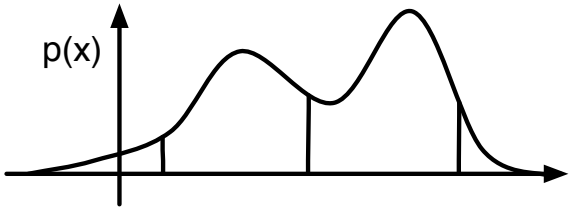

Fig. 1.11. Quantiles of a distribution correspond to the area under the integral of the density $p(x)$ for which the integral takes on a pre-specified value. Illustrated are the 0.1, 0.5 and 0.9 quantiles respectively.

1.2.3 Mean and Variance

A common question to ask about a random variable is what its expected value might be. For instance, when measuring the voltage of a device, we might ask what its typical values might be. When deciding whether to administer a growth hormone to a child a doctor might ask what a sensible range of height should be. For those purposes we need to define expectations and related quantities of distributions.

Definition 1.3 (Mean) *We define the mean of a random variable X as*

$$\mathbb{E}[X] := \int x dp(x) \tag{1.6}$$

More generally, if $f : \mathbb{R} \to \mathbb{R}$ is a function, then $f(X)$ is also a random variable. Its mean is mean given by

$$\mathbb{E}[f(X)] := \int f(x) dp(x). \tag{1.7}$$

Whenever X is a discrete random variable the integral in (1.6) can be replaced by a summation:

$$\mathbb{E}[X] = \sum_x x p(x). \tag{1.8}$$

For instance, in the case of a dice we have equal probabilities of $1/6$ for all 6 possible outcomes. It is easy to see that this translates into a mean of $(1 + 2 + 3 + 4 + 5 + 6)/6 = 3.5$.

The mean of a random variable is useful in assessing expected losses and benefits. For instance, as a stock broker we might be interested in the expected value of our investment in a year's time. In addition to that, however, we also might want to investigate the *risk* of our investment. That is, how likely it is that the value of the investment might deviate from its expectation since this might be more relevant for our decisions. This means that we

need a variable to quantify the risk inherent in a random variable. One such measure is the *variance* of a random variable.

Definition 1.4 (Variance) *We define the variance of a random variable X as*

$$\mathrm{Var}[X] := \mathbb{E}\left[(X - \mathbf{E}[X])^2\right]. \tag{1.9}$$

As before, if $f : \mathbb{R} \to \mathbb{R}$ is a function, then the variance of $f(X)$ is given by

$$\mathrm{Var}[f(X)] := \mathbb{E}\left[(f(X) - \mathbf{E}[f(X)])^2\right]. \tag{1.10}$$

The variance measures by how much on average $f(X)$ deviates from its expected value. As we shall see in Section 2.1, an upper bound on the variance can be used to give guarantees on the probability that $f(X)$ will be within ϵ of its expected value. This is one of the reasons why the variance is often associated with the risk of a random variable. Note that often one discusses properties of a random variable in terms of its *standard deviation*, which is defined as the square root of the variance.

1.2.4 Marginalization, Independence, Conditioning, and Bayes Rule

Given two random variables X and Y, one can write their joint density $p(x, y)$. Given the joint density, one can recover $p(x)$ by integrating out y. This operation is called marginalization:

$$p(x) = \int_y dp(x, y). \tag{1.11}$$

If Y is a discrete random variable, then we can replace the integration with a summation:

$$p(x) = \sum_y p(x, y). \tag{1.12}$$

We say that X and Y are independent, *i.e.,* the values that X takes does not depend on the values that Y takes whenever

$$p(x, y) = p(x)p(y). \tag{1.13}$$

Independence is useful when it comes to dealing with large numbers of random variables whose behavior we want to estimate jointly. For instance, whenever we perform repeated measurements of a quantity, such as when

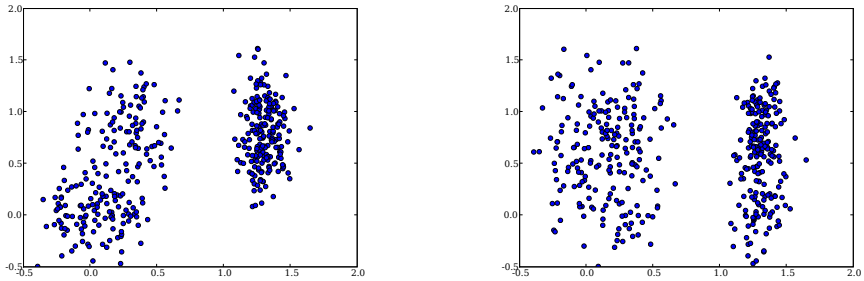

Fig. 1.12. Left: a sample from two dependent random variables. Knowing about first coordinate allows us to improve our guess about the second coordinate. Right: a sample drawn from two independent random variables, obtained by randomly permuting the dependent sample.

measuring the voltage of a device, we will typically assume that the individual measurements are drawn from the same distribution and that they are independent of each other. That is, having measured the voltage a number of times will not affect the value of the next measurement. We will call such random variables to be *independently and identically distributed*, or in short, *iid* random variables. See Figure 1.12 for an example of a pair of random variables drawn from dependent and independent distributions respectively.

Conversely, dependence can be vital in classification and regression problems. For instance, the traffic lights at an intersection are dependent of each other. This allows a driver to perform the inference that when the lights are green in his direction there will be no traffic crossing his path, i.e. the other lights will indeed be red. Likewise, whenever we are given a picture x of a digit, we hope that there will be dependence between x and its label y.

Especially in the case of dependent random variables, we are interested in conditional probabilities, *i.e.,* probability that X takes on a particular value given the value of Y. Clearly $Pr(X = rain|Y = cloudy)$ is higher than $Pr(X = rain|Y = sunny)$. In other words, knowledge about the value of Y significantly influences the distribution of X. This is captured via conditional probabilities:

$$p(x|y) := \frac{p(x, y)}{p(y)}. \tag{1.14}$$

Equation 1.14 leads to one of the key tools in statistical inference.

Theorem 1.5 (Bayes Rule) *Denote by X and Y random variables then*